Roll Number: CB.EN.P2VLD25035, Name: VEDASRI J

Expt No: 2 **Deductive Fault Simulation** **Date:26-02-2026**

Evaluation Table			Faculty Signature
Code:	Output:	Record:	

1. **AIM:** To implement Deductive Fault Simulation for a combinational circuit described in .bench format and compute the set of detectable stuck-at faults for a given test vector.

2. **SOFTWARE/TOOLS USED:** Google Colab (Python)

3. **THEORY:**

3.1 Fault Modeling

In digital circuits, the most commonly used structural fault model is the Single Stuck-At Fault Model.
Each line in the circuit can have two possible faults: Stuck-at-0 (sa0), Stuck-at-1 (sa1).
If a circuit contains N nodes, then:
Total Structural Faults = 2 × N
When fanout branches are modeled explicitly, each branch is treated as an independent node and also contributes two faults.

3.2 Deductive Fault Simulation

Deductive Fault Simulation determines all detectable faults for a given test vector without simulating each fault separately.
For each node:
Fault List at node = Propagated faults from inputs + Local stuck-at fault
The local stuck-at fault added at each node is:
node_sa(1 − true value at node)
Fault lists are propagated using set operations Union and Intersection.

3.3 Deductive Propagation Rules

Let A, B = input nodes, C = output node

Gate type	Inputs a	Inputs b	Output c	Output fault list L_c
AND	0	0	0	$[L_a \cap L_b] \cup c_1$
	0	1	0	$[L_a \cap \overline{L_b}] \cup c_1$
	1	0	0	$[\overline{L_a} \cap L_b] \cup c_1$
	1	1	1	$[L_a \cup L_b] \cup c_0$
OR	0	0	0	$[L_a \cup L_b] \cup c_1$
	0	1	1	$[\overline{L_a} \cap L_b] \cup c_0$
	1	0	1	$[L_a \cap \overline{L_b}] \cup c_0$
	1	1	1	$[L_a \cap L_b] \cup c_0$
NOT	0	-	1	$L_a \cup c_0$
	1	-	0	$L_a \cup c_1$

3.4 Fault Detection

A fault is said to be detected if it appears in the fault list of at least one primary output.
Fault Coverage is calculated as:

$$\text{Coverage} = \frac{\text{Number of Detected Faults}}{\text{Total Structural Faults}} \times 100$$

4. ALGORITHM:

1. Read .bench file
2. Extract Primary inputs, Primary outputs, Gate definitions
3. Expand fanouts into explicit branch nodes
4. Perform topological sorting
5. Apply test vector
6. Perform true value simulation
7. Initialize primary input fault lists
8. Propagate fault lists gate-by-gate
9. Collect detected faults at primary outputs
10. Compute Total faults, Detected faults, Undetected faults, Fault coverage.

5. CODE:

```
#!/usr/bin/env python3

# Import regular expression module for parsing .bench file
import re


# ============================================================
# FUNCTION: load_bench_file
# PURPOSE : Parse .bench file and extract
#           - Primary Inputs
#           - Primary Outputs
#           - Gate definitions
# ============================================================

def load_bench_file(path):

    pi_list = []          # List to store primary input names
    po_list = []          # List to store primary output names
    logic_blocks = {}     # Dictionary to store gate information

    # Open the benchmark file
    with open(path, "r") as fp:
        for raw_line in fp:

            raw_line = raw_line.strip()  # Remove whitespace

            # Skip empty lines or comment lines
            if not raw_line or raw_line.startswith("#"):
                continue

            # Extract Primary Inputs
            if raw_line.startswith("INPUT"):
                match = re.search(r"INPUT\((.*?)\)", raw_line)
                if match:
                    pi_list.append(match.group(1).strip())

            # Extract Primary Outputs
            elif raw_line.startswith("OUTPUT"):
                match = re.search(r"OUTPUT\((.*?)\)", raw_line)
```

```
                if match:
                    po_list.append(match.group(1).strip())




            # Extract Gate Definition
            elif "=" in raw_line:

                # Split output node and gate expression
                left_part, right_part = raw_line.split("=")
                out_net = left_part.strip()

                # Extract gate type and input nodes
                match = re.search(r"([A-Za-z0-9_]+)\((.*?)\)", right_part.strip())

                if match:
                    gate_kind = match.group(1).upper()
                    input_nets = [x.strip() for x in match.group(2).split(",")]

                    # Store gate info in dictionary
                    logic_blocks[out_net] = {
                        "op": gate_kind,
                        "fanin": input_nets
                    }

    return pi_list, po_list, logic_blocks


# ==========================================================
# FUNCTION: create_branch_nodes
# PURPOSE : Explicitly model fanout branches
#           If a net drives multiple gates,
#           create branch nodes (BUFF type)
# ==========================================================

def create_branch_nodes(blocks):

    fanin_usage = {}   # Count how many times each node is used as input

    # Count input usage for each net
    for blk in blocks.values():
        for net in blk["fanin"]:
            fanin_usage[net] = fanin_usage.get(net, 0) + 1

    expanded_blocks = dict(blocks)  # Copy original gates

    # Track branch numbering
    branch_tracker = {
        net: 0 for net, cnt in fanin_usage.items() if cnt > 1
    }

    # Replace multi-fanout connections with explicit branch nodes
    for out_net, blk in list(blocks.items()):

        updated_inputs = []

        for net in blk["fanin"]:

            # If net has multiple fanouts
            if fanin_usage.get(net, 0) > 1:

                branch_tracker[net] += 1
```

```
                # Create new branch node
                branch_net = f"{net}_branch{branch_tracker[net]}"
                updated_inputs.append(branch_net)

                # Branch modeled as BUFF
                expanded_blocks[branch_net] = {
                    "op": "BUFF",
                    "fanin": [net]
                }

            else:
                updated_inputs.append(net)

        expanded_blocks[out_net]["fanin"] = updated_inputs

    return expanded_blocks


# ============================================================
# FUNCTION: generate_evaluation_order
# PURPOSE : Topological sorting of nodes
#           Ensures inputs evaluated before outputs
# ============================================================

def generate_evaluation_order(primary_nodes, blocks):

    ordered_list = list(primary_nodes)  # Start with PIs
    visited_set = set(primary_nodes)    # Mark PIs as visited
    remaining = dict(blocks)            # Gates yet to evaluate

    while remaining:

        progress_flag = False

        for node, blk in list(remaining.items()):

            # If all fanin nodes already evaluated
            if all(inp in visited_set for inp in blk["fanin"]):

                ordered_list.append(node)
                visited_set.add(node)
                del remaining[node]
                progress_flag = True

        # If no progress, there is a loop
        if not progress_flag:
            raise ValueError("Invalid circuit structure (cycle detected).")

    return ordered_list


# ============================================================
# FUNCTION: compute_gate_output
# PURPOSE : Perform true Boolean evaluation
# ============================================================

def compute_gate_output(op_type, inputs):

    if op_type == "AND":
        return 1 if all(inputs) else 0

    if op_type == "NAND":
        return 0 if all(inputs) else 1

    if op_type == "OR":
        return 1 if any(inputs) else 0
```

```
    if op_type == "NOR":
        return 0 if any(inputs) else 1

    if op_type == "XOR":
        return sum(inputs) % 2

    if op_type == "XNOR":
        return 1 - (sum(inputs) % 2)

    if op_type == "NOT":
        return 1 - inputs[0]

    if op_type == "BUFF":
        return inputs[0]

    return inputs[0]


# ============================================================
# FUNCTION: compute_fault_set
# PURPOSE : Deductive Fault Propagation using set theory
# ============================================================

def compute_fault_set(op_type, input_values, input_fault_sets, node_name, node_value):

    # Add own stuck-at fault
    own_fault = f"{node_name}_sa{1 - node_value}"
    propagated_faults = set()

    # NOT / BUFF: propagate input fault list directly
    if op_type in ["NOT", "BUFF"]:
        propagated_faults = set(input_fault_sets[0])

    # XOR / XNOR: union of all input fault sets
    elif op_type in ["XOR", "XNOR"]:
        propagated_faults = set().union(*input_fault_sets)

    # AND / NAND / OR / NOR
    else:
        controlling_value = 0 if op_type in ["AND", "NAND"] else 1

        ctrl_sets = []
        non_ctrl_sets = []

        # Separate controlling and non-controlling inputs
        for val, fset in zip(input_values, input_fault_sets):
            if val == controlling_value:
                ctrl_sets.append(fset)
            else:
                non_ctrl_sets.append(fset)

        if not ctrl_sets:
            # No controlling input
            propagated_faults = set().union(*non_ctrl_sets) if non_ctrl_sets else set()
        else:
            # Intersection of controlling sets
            propagated_faults = set(ctrl_sets[0])
            for s in ctrl_sets[1:]:
                propagated_faults.intersection_update(s)

            # Remove masked faults
            non_ctrl_union = set().union(*non_ctrl_sets) if non_ctrl_sets else set()
            propagated_faults.difference_update(non_ctrl_union)

    propagated_faults.add(own_fault)
    return propagated_faults
```

```
# ============================================================
# MAIN SIMULATION DRIVER
# ============================================================

def run_simulation():

    print("\n===== DEDUCTIVE FAULT SIMULATION =====\n")

    bench_file = input("Enter .bench file name: ")

    try:
        # Parse benchmark file
        pi_list, po_list, raw_blocks = load_bench_file(bench_file)

        # Expand fanouts into branch nodes
        logic_netlist = create_branch_nodes(raw_blocks)

        # Input test vector
        pattern = input(f"Enter test vector ({len(pi_list)} bits): ")

        if len(pattern) != len(pi_list) or not all(bit in "01" for bit in pattern):
            print("Invalid test vector.")
            return

        signal_values = {}       # Stores true logic values
        fault_dictionary = {}    # Stores fault lists

        # Initialize primary inputs
        for net, bit in zip(pi_list, pattern):
            val = int(bit)
            signal_values[net] = val
            fault_dictionary[net] = {f"{net}_sa{1 - val}"}

        # Determine evaluation order
        eval_sequence = generate_evaluation_order(pi_list, logic_netlist)

        # Simulate gates
        for net in eval_sequence:
            if net in pi_list:
                continue

            blk = logic_netlist[net]

            input_vals = [signal_values[x] for x in blk["fanin"]]
            input_faults = [fault_dictionary[x] for x in blk["fanin"]]

            out_val = compute_gate_output(blk["op"], input_vals)
            signal_values[net] = out_val

            fault_dictionary[net] = compute_fault_set(
                blk["op"],
                input_vals,
                input_faults,
                net,
                out_val
            )

        # ================================
        # PRINT NODE TABLE
        # ================================

        print("\n---------------- NODE DETAILS ----------------\n")
        print(f"{'Node':<20}{'Value':<10}Fault Set")
        print("-" * 90)

        for net in eval_sequence:
```

```
            faults_str = ", ".join(sorted(fault_dictionary[net]))
            print(f"{net:<20}{signal_values[net]:<10}{faults_str}")

        # ================================
        # FAULT SUMMARY
        # ================================

        detected = set().union(*(fault_dictionary[po] for po in po_list))

        complete_fault_space = {f"{n}_sa0" for n in eval_sequence} | \
                               {f"{n}_sa1" for n in eval_sequence}

        missed = complete_fault_space - detected

        print("\n---------------- FAULT REPORT ----------------\n")
        print(f"Total Faults       : {len(complete_fault_space)}")
        print(f"Detected Faults    : {len(detected)}")
        print(f"Undetected Faults  : {len(missed)}")
        print(f"Coverage           : {(len(detected)/len(complete_fault_space))*100:.2f}%")

        print("\nDetected Faults:")
        print("{ " + ", ".join(sorted(detected)) + " }")

        print("\nUndetected Faults:")
        print("{ " + ", ".join(sorted(missed)) + " }")

    except Exception as err:
        print("Error:", err)


if __name__ == "__main__":
    run_simulation()
```

6. SAMPLE INPUT:

6.1. Sample circuit

ckt1.bench:

INPUT(1)
INPUT(2)
INPUT(3)
INPUT(4)
INPUT(5)
OUTPUT(9)
6 = AND(2,3)
7 = NOR(4,5)
8 = NAND(6,7)
9 = OR(1,8)

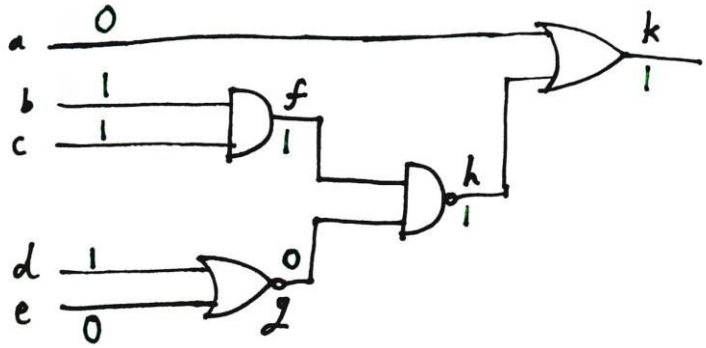

Figure 1: ckt1 circuit diagram

OUTPUT:

```
===== DEDUCTIVE FAULT SIMULATION =====

Enter .bench file name: ckt1.bench
Enter test vector (5 bits): 01110

---------------- NODE DETAILS ----------------

Node                Value     Fault Set
-----------------------------------------------------------------------------------------
---
1                   0         1_sa1
2                   1         2_sa0
3                   1         3_sa0
4                   1         4_sa0
5                   0         5_sa1
6                   1         2_sa0, 3_sa0, 6_sa0
7                   0         4_sa0, 7_sa1
8                   1         4_sa0, 7_sa1, 8_sa0
9                   1         4_sa0, 7_sa1, 8_sa0, 9_sa0

---------------- FAULT REPORT ----------------

Total Faults       : 18
Detected Faults    : 4
Undetected Faults  : 14
Coverage           : 22.22%

Detected Faults:
{ 4_sa0, 7_sa1, 8_sa0, 9_sa0 }

Undetected Faults:
{ 1_sa0, 1_sa1, 2_sa0, 2_sa1, 3_sa0, 3_sa1, 4_sa1, 5_sa0, 5_sa1, 6_sa0, 6_sa1, 7_sa0,
8_sa1, 9_sa1 }
```

6.2. c17

c17.bench:

```
# c17
# 5 inputs
# 2 outputs
# 0 inverter
# 6 gates ( 6 NANDs )

INPUT(1)
INPUT(2)
INPUT(3)
INPUT(6)
INPUT(7)

OUTPUT(22)
OUTPUT(23)
```

10 = NAND(1, 3)
11 = NAND(3, 6)
16 = NAND(2, 11)
19 = NAND(11, 7)
22 = NAND(10, 16)
23 = NAND(16, 19)

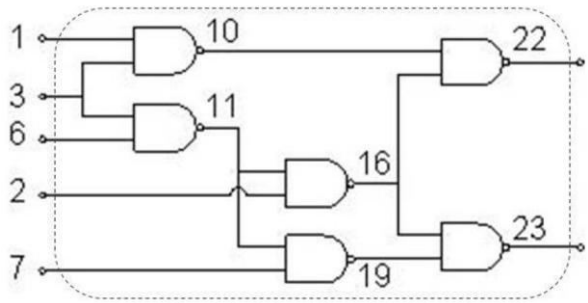

Figure 2: c17 circuit diagram

OUTPUT:

For c17.bench input, with test vector 01010:

```
===== DEDUCTIVE FAULT SIMULATION =====

Enter .bench file name: c17.bench
Enter test vector (5 bits): 01010

---------------- NODE DETAILS ----------------

Node                Value     Fault Set
-------------------------------------------------------------------------------------------
1                   0         1_sa1
2                   1         2_sa0
3                   0         3_sa1
6                   1         6_sa0
7                   0         7_sa1
3_branch1           0         3_branch1_sa1, 3_sa1
3_branch2           0         3_branch2_sa1, 3_sa1
10                  1         10_sa0
11                  1         11_sa0, 3_branch2_sa1, 3_sa1
11_branch1          1         11_branch1_sa0, 11_sa0, 3_branch2_sa1, 3_sa1
11_branch2          1         11_branch2_sa0, 11_sa0, 3_branch2_sa1, 3_sa1
16                  0         11_branch1_sa0, 11_sa0, 16_sa1, 2_sa0, 3_branch2_sa1, 3_sa1
19                  1         19_sa0, 7_sa1
16_branch1          0         11_branch1_sa0, 11_sa0, 16_branch1_sa1, 16_sa1, 2_sa0, 3_branch2_sa1, 3_sa1
16_branch2          0         11_branch1_sa0, 11_sa0, 16_branch2_sa1, 16_sa1, 2_sa0, 3_branch2_sa1, 3_sa1
22                  1         11_branch1_sa0, 11_sa0, 16_branch1_sa1, 16_sa1, 22_sa0, 2_sa0, 3_branch2_sa1, 3_sa1
23                  1         11_branch1_sa0, 11_sa0, 16_branch2_sa1, 16_sa1, 23_sa0, 2_sa0, 3_branch2_sa1, 3_sa1

---------------- FAULT REPORT ----------------

Total Faults       : 34
Detected Faults    : 10
Undetected Faults  : 24
Coverage           : 29.41%

Detected Faults:
{ 11_branch1_sa0, 11_sa0, 16_branch1_sa1, 16_branch2_sa1, 16_sa1, 22_sa0, 23_sa0, 2_sa0, 3_branch2_sa1, 3_sa1 }

Undetected Faults:
{ 10_sa0, 10_sa1, 11_branch1_sa1, 11_branch2_sa0, 11_branch2_sa1, 11_sa1, 16_branch1_sa0, 16_branch2_sa0, 16_sa0,
19_sa0, 19_sa1, 1_sa0, 1_sa1, 22_sa1, 23_sa1, 2_sa1, 3_branch1_sa0, 3_branch1_sa1, 3_branch2_sa0, 3_sa0, 6_sa0, 6_sa1,
7_sa0, 7_sa1}
```

7. INFERENCE:

The deductive fault simulation was successfully implemented for the given combinational circuit. Fault lists were generated for all nodes and propagated to the primary outputs using set-based deductive rules. The detected faults were identified from the primary output fault lists, and fault coverage was computed. It is observed that a single test vector does not detect all structural faults, and multiple vectors are required to improve coverage. The experiment demonstrates that deductive fault simulation efficiently determines detectable faults in a single pass without individually simulating each fault.